

SVMs, the Kernel Trick

MFML Sec9

August 29, 2025

Contents

1	The Task and the Classifier	2
2	Margin From First Principles (Three Parallel Planes)	2
2.1	Picture and Setup	2
2.2	Shortest Direction and the $x = k w$ Trick	2
2.3	Compute the Margin and the Origin–Plane Distance	2
3	Primal SVM: Hard and Soft Margin	3
3.1	Hard Margin (Linearly Separable Case)	3
3.2	Soft Margin (Non-separable Case)	3
4	Understanding Hinge Loss	3
5	The Dual: Full Algebra With No Gaps	4
5.1	Hard-Margin Dual	4
5.2	Soft-Margin Dual and the Box Constraint $0 \leq \alpha_i \leq C$	5
5.3	Recovering w and b ; KKT Interpretation	5
6	Mapping to a Feature Space and What a Kernel Is	6
6.1	The Motivation: When Linear Isn't Enough	6
6.2	The Computational Problem: Why We Need a Shortcut	6
6.3	The Kernel Trick: The Elegant Shortcut	6
6.4	Common Kernels	7
7	Deeper Dive: Feature Space Construction	7
7.1	Polynomial Features and Exact Scaling (Why $\sqrt{\text{coeff}}$ Appears)	7
7.2	Counting Features: From Few to Many Dimensions	8
8	RBF Kernel = Infinite-Dimensional Feature Space	8
9	Worked Example (XOR) Solved Completely by Hand	9
9.1	Data and Labels	9
9.2	Kernel Matrix for Degree-2 Polynomial $K(x, z) = (x \cdot z)^2$	9
9.3	Dual Optimization (Hard Margin) With Symmetry	9
9.4	Compute b Exactly (KKT Tightness)	10
9.5	Final Decision Function and Verification	10
9.6	Recover the Explicit Separator in Feature Space	10

1 The Task and the Classifier

WE have training data $\{(x_i, y_i)\}_{i=1}^m$ with data points $x_i \in \mathbb{R}^n$ and corresponding class labels $y_i \in \{-1, +1\}$. A linear classifier aims to separate these points by predicting the label of a new point x using the function:

$$f(x) = \text{sign}(w \cdot x + b),$$

where $w \in \mathbb{R}^n$ is a weight vector and $b \in \mathbb{R}$ is a bias term. The equation $w \cdot x + b = 0$ defines a hyperplane. This hyperplane is our **decision boundary**; it splits the n -dimensional space into two half-spaces, one for each class.

2 Margin From First Principles (Three Parallel Planes)

2.1 Picture and Setup

The core idea of SVM is not just to separate the data, but to do so with the largest possible “cushion” or **margin**. We can visualize this with three parallel planes, all having the same normal vector w :

$$\text{Negative Support Plane: } w \cdot x + b = -1$$

$$\text{Decision Plane: } w \cdot x + b = 0$$

$$\text{Positive Support Plane: } w \cdot x + b = +1$$

The margin is the perpendicular distance between the two support planes. We will demand that all positive examples ($y_i = +1$) lie on one side of the positive support plane, and all negative examples ($y_i = -1$) on the other side of the negative support plane.

2.2 Shortest Direction and the $x = k w$ Trick

To find the distance between parallel planes, we need to move along the direction perpendicular to them. This direction is given by the normal vector, w . So, we look for points on these planes that lie on the line $\{k w : k \in \mathbb{R}\}$ passing through the origin.

For a target value $c \in \{-1, 0, +1\}$, we substitute $x = k w$ into the plane equation $w \cdot x + b = c$:

$$w \cdot (k w) + b = c \Rightarrow k (w \cdot w) + b = c \Rightarrow k \|w\|^2 + b = c \Rightarrow k = \frac{c - b}{\|w\|^2}.$$

This gives us the specific points on each plane that are closest to the origin:

$$x_- = \frac{-1 - b}{\|w\|^2} w, \quad x_0 = \frac{-b}{\|w\|^2} w, \quad x_+ = \frac{1 - b}{\|w\|^2} w.$$

2.3 Compute the Margin and the Origin–Plane Distance

The margin is the distance between x_+ and x_- . The distance from the origin to the decision plane is the length of the vector x_0 .

$$\text{margin} = \|x_+ - x_-\| = \left\| \frac{1 - b}{\|w\|^2} w - \frac{-1 - b}{\|w\|^2} w \right\| = \left| \frac{1 - b - (-1 - b)}{\|w\|^2} \right| \|w\| = \frac{2}{\|w\|^2} \|w\| = \frac{2}{\|w\|},$$

$$\text{dist}(0, w \cdot x + b = 0) = \|x_0\| = \left| \frac{-b}{\|w\|^2} \right| \|w\| = \frac{|b|}{\|w\|}.$$

Why SVM fixes the scale. Notice that if we scale (w, b) to $(\lambda w, \lambda b)$ for some $\lambda > 0$, the decision boundary $w \cdot x + b = 0$ remains unchanged. However, the margin $2/\|w\|$ changes. To make the problem well-defined, SVM *fixes the scale* by enforcing the constraint that for all data points, the value of $w \cdot x_i + b$ is at least 1 (for positive points) or at most -1 (for negative points). This can be written compactly as:

$$y_i(w \cdot x_i + b) \geq 1 \quad \forall i.$$

This normalization ensures that the closest data points (the “support vectors”) lie exactly on the support planes $w \cdot x + b = \pm 1$. With this rule, maximizing the margin $2/\|w\|$ is the same as minimizing $\|w\|$, or for convenience, minimizing $\frac{1}{2}\|w\|^2$.

3 Primal SVM: Hard and Soft Margin

3.1 Hard Margin (Linearly Separable Case)

For data that can be perfectly separated by a hyperplane, the optimization problem is:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 && \text{(Maximize margin)} \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1, \quad i = 1, \dots, m. && \text{(All points correctly classified)} \end{aligned}$$

3.2 Soft Margin (Non-separable Case)

Real-world data is often not perfectly separable. To handle this, we introduce non-negative *slack variables* $\xi_i \geq 0$. A point x_i can now be inside the margin ($\xi_i > 0$) or even on the wrong side of the hyperplane ($\xi_i > 1$), but we pay a penalty for it. The problem becomes:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C \sum_{i=1}^m \xi_i && \text{(Maximize margin AND minimize error)} \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i, && \text{(Constraints are now “soft”)} \\ & \xi_i \geq 0. && \end{aligned}$$

The parameter $C > 0$ is a hyperparameter that controls the trade-off. A large C heavily penalizes constraint violations, approaching the hard-margin case. A small C allows for a wider margin at the cost of more violations.

4 Understanding Hinge Loss

The soft-margin SVM formulation is directly connected to a concept called **hinge loss**. Let’s see how. From the constraint $y_i(w \cdot x_i + b) \geq 1 - \xi_i$, we can write $\xi_i \geq 1 - y_i(w \cdot x_i + b)$. Since we also have $\xi_i \geq 0$ and the objective function aims to minimize $\sum \xi_i$, we will choose the smallest possible value for each ξ_i that satisfies both conditions. This means:

$$\xi_i = \max \{0, 1 - y_i(w \cdot x_i + b)\}.$$

Substituting this back into the soft-margin objective gives an unconstrained optimization problem:

$$\min_{w,b} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \max \{0, 1 - y_i(w \cdot x_i + b)\}.$$

The term $\max\{0, 1 - z\}$ is the **hinge loss function**, where $z = y_i(w \cdot x_i + b)$ is the “score” of a data point.

Intuition. The hinge loss has a simple, powerful meaning:

- If $y_i(w \cdot x_i + b) \geq 1$: The point is correctly classified and outside the margin. The score $z \geq 1$, so the loss $\max\{0, 1 - z\}$ is **0**. The model is not penalized.
- If $y_i(w \cdot x_i + b) < 1$: The point is either inside the margin or misclassified. The score $z < 1$, and the loss is $1 - z$, a **linear penalty** that grows the further the point is from the correct margin boundary.

So, SVM tries to find a balance between making the margin large (small $\|w\|^2$) and ensuring most points have a score of at least 1 (small total hinge loss).

5 The Dual: Full Algebra With No Gaps

We now transform the primal problem into its dual form. This seems like a detour, but it’s the key that unlocks the kernel trick. The dual formulation will reveal that the data points x_i are only ever used in the form of dot products, $x_i \cdot x_j$.

5.1 Hard-Margin Dual

We start with the Lagrangian for the hard-margin problem, introducing a Lagrange multiplier $\alpha_i \geq 0$ for each of the m constraints.

$$\begin{aligned} \mathcal{L}(w, b, \alpha) &= (\text{Objective}) - \sum (\text{Multiplier} \times \text{Constraint}) \\ &= \frac{1}{2} w \cdot w - \sum_{i=1}^m \alpha_i (y_i(w \cdot x_i + b) - 1) \\ &= \frac{1}{2} w \cdot w - \sum_i \alpha_i y_i (w \cdot x_i) - b \sum_i \alpha_i y_i + \sum_i \alpha_i. \end{aligned} \quad (1)$$

To find the minimum of $\mathcal{L}$ with respect to w and b , we set the partial derivatives to zero (stationarity conditions):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w} = 0 &\Rightarrow w - \sum_i \alpha_i y_i x_i = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i, \\ \frac{\partial \mathcal{L}}{\partial b} = 0 &\Rightarrow - \sum_i \alpha_i y_i = 0 \Rightarrow \sum_i \alpha_i y_i = 0. \end{aligned}$$

Now we substitute these two results back into the Lagrangian $\mathcal{L}$ to get the dual objective, which depends only on α .

- **Term 1:** $\frac{1}{2} w \cdot w$. Substitute w :

$$\frac{1}{2} w \cdot w = \frac{1}{2} \left(\sum_i \alpha_i y_i x_i \right) \cdot \left(\sum_j \alpha_j y_j x_j \right) = \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j).$$

- **Term 2:** $- \sum_i \alpha_i y_i (w \cdot x_i)$. Substitute w :

$$- \sum_i \alpha_i y_i \left(\left(\sum_j \alpha_j y_j x_j \right) \cdot x_i \right) = - \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_j \cdot x_i) = - \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j).$$

- **Term 3:** $-b \sum_i \alpha_i y_i$. From stationarity, $\sum_i \alpha_i y_i = 0$, so this term is zero.
- **Term 4:** $+\sum_i \alpha_i$. This term remains as is.

Combining these terms, the dual objective is (Term 1 + Term 2 + Term 3 + Term 4):

$$\mathcal{L}(\alpha) = \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + 0 + \sum_i \alpha_i = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j).$$

The dual problem is to maximize this objective subject to the constraints we found:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0. \end{aligned}$$

5.2 Soft-Margin Dual and the Box Constraint $0 \leq \alpha_i \leq C$

The derivation is very similar. We add multipliers $\mu_i \geq 0$ for the slack constraints $\xi_i \geq 0$:

$$\mathcal{L} = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i (y_i (w \cdot x_i + b) - 1 + \xi_i) - \sum_i \mu_i \xi_i.$$

The derivatives with respect to w and b give the same conditions as before. The new derivative is with respect to ξ_i :

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \Rightarrow C - \alpha_i - \mu_i = 0 \Rightarrow \mu_i = C - \alpha_i.$$

Since we require $\mu_i \geq 0$ and we already have $\alpha_i \geq 0$, this implies $C - \alpha_i \geq 0$, which gives us the famous *box constraint*: $0 \leq \alpha_i \leq C$. The dual problem is identical to the hard-margin case, but with this updated constraint.

5.3 Recovering w and b ; KKT Interpretation

After solving the dual problem for the optimal α_i 's, we can recover w and b . The formula for w is the same as before: $w = \sum_i \alpha_i y_i x_i$.

To find b , we use the Karush-Kuhn-Tucker (KKT) complementary slackness conditions. For the soft-margin case, one of these conditions is:

$$\alpha_i (y_i (w \cdot x_i + b) - 1 + \xi_i) = 0.$$

For a data point x_s where $0 < \alpha_s < C$ (a true margin support vector), the condition implies that $\xi_s = 0$ and the term in the parenthesis must be zero:

$$y_s (w \cdot x_s + b) - 1 = 0 \Rightarrow y_s (w \cdot x_s + b) = 1.$$

Multiplying by y_s (note $y_s^2 = 1$), we get $w \cdot x_s + b = y_s$. We can now solve for b :

$$b = y_s - w \cdot x_s \Rightarrow b = y_s - \left(\sum_j \alpha_j y_j x_j \right) \cdot x_s \Rightarrow b = y_s - \sum_j \alpha_j y_j (x_j \cdot x_s).$$

The final prediction for a new point x depends only on dot products with the support vectors (those x_i with $\alpha_i > 0$):

$$f(x) = \text{sign} \left(\sum_i \alpha_i y_i (x_i \cdot x) + b \right).$$

6 Mapping to a Feature Space and What a Kernel Is

6.1 The Motivation: When Linear Isn't Enough

What if our data isn't linearly separable? Consider the XOR problem, where points are arranged in a checkerboard pattern. No single line can separate them. The trick is to project the data into a higher-dimensional space where it *does* become linearly separable.

We define a **feature map** $\phi(\cdot)$ that takes a point $x \in \mathbb{R}^n$ and maps it to a new point $\phi(x) \in \mathbb{R}^p$ in a higher-dimensional feature space (where $p > n$).

$$\phi : \mathbb{R}^n \rightarrow \mathbb{R}^p, \quad x \mapsto \phi(x) = (\phi_1(x), \dots, \phi_p(x)).$$

In this new space, we can find a linear decision boundary $W \cdot \phi(x) + b = 0$. When mapped back to the original space, this linear boundary in $\mathbb{R}^p$ becomes a complex, non-linear boundary in $\mathbb{R}^n$.

6.2 The Computational Problem: Why We Need a Shortcut

Let's consider a simple example. Let our input space be $\mathbb{R}^2$ and we want to create a degree-2 polynomial feature space. A possible feature map is:

$$x = (x_1, x_2) \mapsto \phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, 1) \in \mathbb{R}^6.$$

Now, look at the dual SVM formulation. It contains the term $(x_i \cdot x_j)$. If we use our feature map, we must replace this with $\phi(x_i) \cdot \phi(x_j)$. To calculate this for two points x_i and x_j , we would have to:

1. **Explicitly compute** $\phi(x_i)$: This involves 6 calculations.
2. **Explicitly compute** $\phi(x_j)$: Another 6 calculations.
3. **Compute their dot product**: This involves 6 multiplications and 5 additions.

This is tedious even for this simple case. If our original dimension $n = 100$ and our polynomial degree $d = 3$, the feature space dimension p becomes huge (thousands of dimensions), and this direct approach becomes computationally impossible.

6.3 The Kernel Trick: The Elegant Shortcut

A **kernel** is a function $K(x, z)$ that gives the same result as the dot product in the feature space, but without ever having to compute the ϕ vectors explicitly.

$$K(x, z) = \phi(x) \cdot \phi(z)$$

This is the famous **kernel trick**. Let's revisit our polynomial example. Consider the kernel function $K(x, z) = (x \cdot z + 1)^2$. For $x = (x_1, x_2)$ and $z = (z_1, z_2)$:

$$\begin{aligned} K(x, z) &= (x_1z_1 + x_2z_2 + 1)^2 \\ &= (x_1z_1)^2 + (x_2z_2)^2 + 1^2 + 2x_1z_1x_2z_2 + 2x_1z_1 + 2x_2z_2 \\ &= (x_1^2)(z_1^2) + (x_2^2)(z_2^2) + (\sqrt{2}x_1)(\sqrt{2}z_1) + (\sqrt{2}x_2)(\sqrt{2}z_2) + (\sqrt{2}x_1x_2)(\sqrt{2}z_1z_2) + (1)(1). \end{aligned}$$

This is exactly the dot product of $\phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, 1)$ with the corresponding $\phi(z)$.

Long Way vs. Shortcut Comparison.

- **Long Way (Explicit Mapping):** Compute two 6-dimensional vectors and then their dot product (many operations).
- **Shortcut (Kernel Trick):** Compute the simple dot product $x \cdot z$ in 2D (2 multiplications, 1 addition), add 1, and square the result (1 multiplication).

The kernel gives us the power of operating in a high-dimensional space while only paying the computational price of the original, low-dimensional space. We simply replace every instance of $(x_i \cdot x_j)$ in the dual SVM formulation with $K(x_i, x_j)$.

Similarity Intuition. A kernel $K(x, z)$ can be thought of as a measure of similarity between points x and z . A large value means the points are “similar” according to the features implicitly defined by the kernel.

6.4 Common Kernels

- **Linear:** $K(x, z) = x \cdot z$. This is just the standard SVM with no feature lift.
- **Polynomial:** $K(x, z) = (x \cdot z + c)^d$. This implicitly creates a feature space of all polynomial terms up to degree d .
- **RBF/Gaussian:** $K(x, z) = \exp(-\gamma \|x - z\|^2)$. This is a very powerful kernel that corresponds to an infinite-dimensional feature space.

7 Deeper Dive: Feature Space Construction

7.1 Polynomial Features and Exact Scaling (Why $\sqrt{\text{coeff}}$ Appears)

Let’s understand where the $\sqrt{2}$ in our earlier example came from. Our goal is to find a ϕ such that its dot product *exactly* reproduces a kernel function. Let’s use the pure polynomial kernel $K(x, z) = (x \cdot z)^2$ for $x, z \in \mathbb{R}^2$.

First, expand the kernel algebraically:

$$(x \cdot z)^2 = (x_1 z_1 + x_2 z_2)^2 = (x_1 z_1)^2 + 2(x_1 z_1)(x_2 z_2) + (x_2 z_2)^2 = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2.$$

We need to define a feature vector $\phi(x)$ such that when we compute $\phi(x) \cdot \phi(z)$, we get exactly this expression. We can achieve this by grouping the x and z terms:

$$\phi(x) \cdot \phi(z) = (\text{term for } x_1^2)(\text{term for } z_1^2) + (\text{term for } x_1 x_2)(\text{term for } z_1 z_2) + (\text{term for } x_2^2)(\text{term for } z_2^2).$$

Matching this to the expansion above:

- To get $x_1^2 z_1^2$, the first components of $\phi(x)$ and $\phi(z)$ must be x_1^2 and z_1^2 .
- To get $x_2^2 z_2^2$, the last components must be x_2^2 and z_2^2 .
- To get the middle term, $2x_1 x_2 z_1 z_2$, the product of the middle components must equal this. We can choose the components to be $\sqrt{2} x_1 x_2$ and $\sqrt{2} z_1 z_2$, because their product is $(\sqrt{2})(\sqrt{2})x_1 x_2 z_1 z_2 = 2x_1 x_2 z_1 z_2$.

This gives us the required feature map:

$$\phi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2).$$

The square root of the multinomial coefficient appears so that when the components are multiplied, they reconstruct the original integer coefficient from the expansion. For a general polynomial kernel $(x \cdot z)^d$, the multinomial theorem gives the coefficients for the expansion, and the feature map scales each monomial term by the square root of its corresponding coefficient.

7.2 Counting Features: From Few to Many Dimensions

Where does the formula for the number of features come from? This is a classic combinatorics problem called “stars and bars”.

Problem: How many unique monomials of exactly degree d can be formed from n variables $(x_1, \dots, x_n)$? A monomial is of the form $x_1^{k_1} x_2^{k_2} \dots x_n^{k_n}$ where the exponents must sum to d : $k_1 + k_2 + \dots + k_n = d$.

Intuition (Stars and Bars): Imagine we have d “stars” (★) representing the total degree. We need to divide these stars among n “bins” (the variables $x_1, \dots, x_n$). To do this, we can place $n - 1$ “bars” (|) among the stars. For example, if $n = 4, d = 5$, the arrangement

$$\star \star \mid \star \mid \star \star \mid$$

means $k_1 = 2, k_2 = 1, k_3 = 2, k_4 = 0$. The total number of items (stars and bars) is $d + (n - 1)$. The number of ways to arrange them is the number of ways to choose the positions for the d stars out of the $d + n - 1$ total positions. This is given by the binomial coefficient:

$$\text{Number of features (exactly degree } d) = \binom{d + n - 1}{d}.$$

For features of degree up to d , the formula changes slightly to $\binom{n+d}{d}$.

Example. For $n = 10$ variables and degree $d = 4$:

$$\text{Features of exactly degree 4: } \binom{10 + 4 - 1}{4} = \binom{13}{4} = 715.$$

Even for modest inputs, the feature space dimension explodes, reinforcing the need for the kernel trick.

8 RBF Kernel = Infinite-Dimensional Feature Space

The Gaussian or RBF kernel is defined as $K(x, z) = \exp(-\gamma \|x - z\|^2)$. It’s not obvious that this corresponds to a dot product. Let’s prove it with algebra. First, expand the norm: $\|x - z\|^2 = \|x\|^2 + \|z\|^2 - 2x \cdot z$.

$$K(x, z) = e^{-\gamma(\|x\|^2 + \|z\|^2 - 2x \cdot z)} = e^{-\gamma\|x\|^2} e^{-\gamma\|z\|^2} e^{2\gamma x \cdot z}.$$

Now, use the Taylor series expansion for the exponential function, $e^t = \sum_{k=0}^{\infty} \frac{t^k}{k!}$, on the last term:

$$e^{2\gamma x \cdot z} = \sum_{k=0}^{\infty} \frac{(2\gamma x \cdot z)^k}{k!} = \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (x \cdot z)^k.$$

As we saw earlier, each term $(x \cdot z)^k$ is itself a dot product in a feature space containing all monomials of degree k . So, we can write:

$$K(x, z) = \underbrace{\left(e^{-\gamma\|x\|^2} \sum_{k=0}^{\infty} \sqrt{\frac{(2\gamma)^k}{k!}} \phi_k(x) \right)}_{\text{This is } \phi(x)} \cdot \underbrace{\left(e^{-\gamma\|z\|^2} \sum_{k=0}^{\infty} \sqrt{\frac{(2\gamma)^k}{k!}} \phi_k(z) \right)}_{\text{This is } \phi(z)}.$$

The implicit feature map $\phi(x)$ is an infinite-dimensional vector, containing weighted versions of all monomial features of every possible degree. The magic is that we can compute this infinite-dimensional dot product by simply calculating one exponential, completely avoiding the impossible task of constructing the ϕ vector.

9 Worked Example (XOR) Solved Completely by Hand

9.1 Data and Labels

Let's solve the XOR problem. The data is not linearly separable in $\mathbb{R}^2$.

i	$x_i = (x_{i1}, x_{i2})$	y_i
1	(1, 1)	-1
2	(1, -1)	+1
3	(-1, 1)	+1
4	(-1, -1)	-1

The pattern is: if the signs of the components are the same, $y = -1$; if they are different, $y = +1$.

9.2 Kernel Matrix for Degree-2 Polynomial $K(x, z) = (x \cdot z)^2$

We will use a simple degree-2 polynomial kernel. First, let's compute all pairwise dot products $x_i \cdot x_j$. The matrix of dot products is:

$$x_i \cdot x_j = \begin{pmatrix} 2 & 0 & 0 & -2 \\ 0 & 2 & -2 & 0 \\ 0 & -2 & 2 & 0 \\ -2 & 0 & 0 & 2 \end{pmatrix}$$

Now we compute the kernel matrix $K_{ij} = (x_i \cdot x_j)^2$ by squaring each element:

$$K = \begin{bmatrix} 4 & 0 & 0 & 4 \\ 0 & 4 & 4 & 0 \\ 0 & 4 & 4 & 0 \\ 4 & 0 & 0 & 4 \end{bmatrix}.$$

9.3 Dual Optimization (Hard Margin) With Symmetry

The hard-margin dual problem is:

$$\max_{\alpha \geq 0} \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{i,j=1}^4 \alpha_i \alpha_j y_i y_j K_{ij} \quad \text{s.t.} \quad \sum_{i=1}^4 \alpha_i y_i = 0.$$

The problem is symmetric. Points x_1 and x_4 have label $y = -1$, and points x_2 and x_3 have label $y = +1$. It is reasonable to assume that the solution will also be symmetric, so let's set $\alpha_1 = \alpha_4 = a$ and $\alpha_2 = \alpha_3 = b$. The equality constraint $\sum \alpha_i y_i = 0$ becomes:

$$\alpha_1 y_1 + \alpha_2 y_2 + \alpha_3 y_3 + \alpha_4 y_4 = 0 \Rightarrow a(-1) + b(+1) + b(+1) + a(-1) = 0 \Rightarrow -2a + 2b = 0 \Rightarrow a = b.$$

So all α_i must be equal! Let's call this common value a . The objective function $L(\alpha)$ becomes a function of a :

$$L(a) = \sum_{i=1}^4 a - \frac{1}{2} \sum_{i,j=1}^4 a \cdot a \cdot y_i y_j K_{ij} = 4a - \frac{a^2}{2} \sum_{i,j=1}^4 y_i y_j K_{ij}.$$

Let's compute the big sum $S = \sum y_i y_j K_{ij}$. This can be written as $y^T K y$:

$$S = \begin{pmatrix} -1 & 1 & 1 & -1 \end{pmatrix} \begin{bmatrix} 4 & 0 & 0 & 4 \\ 0 & 4 & 4 & 0 \\ 0 & 4 & 4 & 0 \\ 4 & 0 & 0 & 4 \end{bmatrix} \begin{pmatrix} -1 \\ 1 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 & 1 & 1 & -1 \end{pmatrix} \begin{pmatrix} -8 \\ 8 \\ 8 \\ -8 \end{pmatrix} = 8 + 8 + 8 + 8 = 32.$$

Our objective function simplifies to $L(a) = 4a - \frac{a^2}{2}(32) = 4a - 16a^2$. To maximize this, we take the derivative and set it to zero:

$$L'(a) = 4 - 32a = 0 \Rightarrow a = \frac{4}{32} = \frac{1}{8}.$$

(The second derivative $L''(a) = -32 < 0$, confirming this is a maximum). So, the optimal solution is $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{8}$. Since all $\alpha_i > 0$, all four points are support vectors.

9.4 Compute b Exactly (KKT Tightness)

We can find b using any support vector x_s (here, any point will do). The formula is $b = y_s - \sum_{j=1}^4 \alpha_j y_j K_{sj}$. Let's use $s = 1$:

$$b = y_1 - \sum_{j=1}^4 \alpha_j y_j K_{1j} = y_1 - (\alpha_1 y_1 K_{11} + \alpha_2 y_2 K_{12} + \alpha_3 y_3 K_{13} + \alpha_4 y_4 K_{14}).$$

Plugging in the values ($y_1 = -1$, $\alpha_j = 1/8$, and the first row of K is $(4, 0, 0, 4)$):

$$\sum_j \alpha_j y_j K_{1j} = \frac{1}{8} [(-1)(4) + (+1)(0) + (+1)(0) + (-1)(4)] = \frac{1}{8} [-4 - 4] = -1.$$

Therefore, $b = y_1 - (-1) = -1 + 1 = 0$. By symmetry, we would get $b = 0$ using any other point.

9.5 Final Decision Function and Verification

The decision score for a new point x is $g(x) = \sum_i \alpha_i y_i K(x_i, x) + b$. Let's test it on one of our training points, $x_2 = (1, -1)$. We need the kernel values $K(x_i, x_2)$, which is the second column of K : $(0, 4, 4, 0)$.

$$\begin{aligned} g(x_2) &= \sum_{i=1}^4 \alpha_i y_i K(x_i, x_2) + 0 \\ &= \frac{1}{8} [y_1 K(x_1, x_2) + y_2 K(x_2, x_2) + y_3 K(x_3, x_2) + y_4 K(x_4, x_2)] \\ &= \frac{1}{8} [(-1) \cdot 0 + (+1) \cdot 4 + (+1) \cdot 4 + (-1) \cdot 0] = \frac{1}{8} [8] = +1. \end{aligned}$$

The prediction is $\text{sign}(g(x_2)) = \text{sign}(+1) = +1$, which matches the true label $y_2 = +1$. You can verify that it works for all four points.

9.6 Recover the Explicit Separator in Feature Space

For our kernel $K(x, z) = (x \cdot z)^2$ in 2D, the corresponding feature map is $\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$. We can find the explicit weight vector W in this feature space using $W = \sum_i \alpha_i y_i \phi(x_i)$:

$$\begin{aligned} W &= \frac{1}{8} y_1 \phi(x_1) + \frac{1}{8} y_2 \phi(x_2) + \frac{1}{8} y_3 \phi(x_3) + \frac{1}{8} y_4 \phi(x_4) \\ &= \frac{1}{8} [(-1)\phi(1, 1) + (+1)\phi(1, -1) + (+1)\phi(-1, 1) + (-1)\phi(-1, -1)] \\ &= \frac{1}{8} [(-1)(1, \sqrt{2}, 1) + (1)(1, -\sqrt{2}, 1) + (1)(1, -\sqrt{2}, 1) + (-1)(1, \sqrt{2}, 1)] \\ &= \frac{1}{8} [(-1 + 1 + 1 - 1), (-\sqrt{2} - \sqrt{2} - \sqrt{2} - \sqrt{2}), (-1 + 1 + 1 - 1)] \\ &= \frac{1}{8} (0, -4\sqrt{2}, 0) = \left(0, -\frac{\sqrt{2}}{2}, 0\right) = \left(0, -\frac{1}{\sqrt{2}}, 0\right). \end{aligned}$$

The decision boundary in feature space is $W \cdot \phi(x) + b = 0$.

$$\left(0, -\frac{1}{\sqrt{2}}, 0\right) \cdot (x_1^2, \sqrt{2}x_1x_2, x_2^2) + 0 = 0 \Rightarrow -\frac{1}{\sqrt{2}}(\sqrt{2}x_1x_2) = 0 \Rightarrow -x_1x_2 = 0.$$

The final classifier is $\text{sign}(-x_1x_2)$.

- For $(1, 1)$ or $(-1, -1)$, $x_1x_2 = 1$, prediction is $\text{sign}(-1) = -1$. Correct.
- For $(1, -1)$ or $(-1, 1)$, $x_1x_2 = -1$, prediction is $\text{sign}(+1) = +1$. Correct.

The curved decision boundary $x_1x_2 = 0$ (which corresponds to the axes) perfectly separates the XOR data.